

39. **Паттерны проектирования. Виды паттернов проектирования. Основные идеи и задачи их применения**

Лекция не затрагивает эту тему, но она является краеугольным камнем современной разработки.

Паттерны проектирования (Design Patterns) — это типовые, проверенные способы решения часто встречающихся проблем при проектировании классов и объектов на уровне исходного кода. Это более «человеческие» решения, чем архитектурные шаблоны.

Основные цели применения:

Ускорить разработку, предлагая готовые, отлаженные решения.

Повысить читаемость и поддерживаемость кода, так как паттерны являются стандартными «языком» разработчиков.

Снизить количество ошибок, которые могут возникнуть при «изобретении велосипеда».

Виды паттернов (по книге «Банды четырех» — GoF):

Порождающие (Creational): Решают проблемы создания объектов (например, Singleton, Factory, Builder).

Структурные (Structural): Описывают, как классы и объекты объединяются в более крупные структуры (например, Adapter, Decorator, Facade).

Поведенческие (Behavioral): Определяют эффективные механизмы взаимодействия между объектами и распределения ответственности (например, Observer, Strategy, Command).

40. **Архитектурные шаблоны MV***

Лекция не содержит этой информации. Архитектурные шаблоны семейства MV* строятся вокруг разделения данных (Model), их представления (View) и логики их обработки (последний компонент).

MVC (Model-View-Controller): Классический шаблон. Controller получает запрос от пользователя, работает с Model (бизнес-логика и данные) и выбирает View для отображения результата.

MVP (Model-View-Presenter): Эволюция MVC. Presenter выступает посредником между View и Model, полностью обрабатывая логику представления. View становится пассивным («тонким клиентом»).

MVVM (Model-View-View-Model): Активно используется в технологиях с привязкой данных (data binding). ViewModel преобразует данные из Model в команды и свойства, с которыми автоматически синхронизируется View.

Эти шаблоны являются практической реализацией «принципа многофункциональности» и «принципа логической независимости», так как они разделяют модель данных (логику) и её отображение, позволяя изменять их независимо.

41. **Архитектура корпоративных программных приложений. Современные подходы в организации ИС**

Лекция описывает классический подход (модели AS-IS, TO-BE, затем проектирование ИС). Современные же подходы эволюционировали в сторону большей гибкости.

Отход от больших монолитных релизов.

Служба-ориентированная архитектура (SOA): Система строится как набор взаимодействующих через сеть сервисов.

Микросервисная архитектура: Эволюция SOA.

Событийно-ориентированная архитектура (EDA): Компоненты системы реагируют на события, что повышает масштабируемость и отказоустойчивость.

Бессерверные вычисления (Serverless).

42. **Архитектура микросервисов (Microservices Architecture)**

Это один из главных современных подходов.

Определение: Подход, при котором одно приложение строится как набор небольших, независимо разрабатываемых сервисов. Каждый сервис реализует определенную бизнес-функцию.

Характеристики:

Независимость: Сервисы могут разрабатываться, деплоиться и масштабироваться разными командами с использованием разных технологий. Ограниченный контекст: Каждый сервис владеет своей бизнес-логикой и базой данных.

Устойчивость: Отказ одного сервиса не приводит к падению всего приложения.

Связь с лекциями: Это высшая степень реализации «принципа декомпозиции» на уровне целого приложения.

43. **Бессерверная архитектура (Serverless Architecture)**

Определение: Модель развертывания и исполнения кода, при которой разработчик не управляет серверами. Облачный провайдер (AWS Lambda, Google Cloud Functions) полностью берёт на себя запуск, масштабирование и обслуживание инфраструктуры.

Принцип работы: Разработчик загружает в облако функцию (кусочек кода). Платформа запускает её только при наступлении определённого события (например, HTTP-запроса) и автоматически выделяет под неё столько ресурсов, сколько нужно.

Связь с лекциями: Это экстремальная степень «принципа абстрагирования», где управление инфраструктурой полностью скрыто от разработчика, и он фокусируется исключительно на коде.

44. **Понятие «рефакторинга». Методы рефакторинга**

Рефакторинг (Refactoring) — это процесс изменения внутренней структуры программного кода без изменения его внешнего поведения. Цель — улучшить читаемость, понятность и снизить сложность кода, облегчить его поддержку и последующее развитие.

Методы рефакторинга (основаны на книге Мартина Фаулера):

Извлечение метода (Extract Method): Превращение фрагмента кода в отдельный метод.

Переименование (Rename): Изменение названия переменной, метода, класса на более понятное.

Удаление дублирования (Remove Duplication): Объединение одинаковых фрагментов кода в одном месте.

Инкапсуляция полей (Encapsulate Field): Создание геттеров и сеттеров для приватного доступа к данным.

Замена условного оператора полиморфизмом (Replace Conditional with Polymorphism): Использование наследования вместо

словных if/else или switch.